

WhereHouse

Codex + VS Code Development Handoff

Know what you have. Know where it is.

Repository	github.com/Jamesbillard12/wherehouse
Current feature branch	feature/core-domain
Open pull request	#1 - Build initial core domain backend
Purpose	Project context for continuing development with Codex inside VS Code

1. Product overview

WhereHouse is a household inventory and storage platform designed to answer six practical questions: What do I have? Where is it? Where should it go? Who has it right now? What did I move? What do I need for this activity?

- Track household items and their exact physical storage locations.
- Model areas, zones, containers, nested containers, and item placement.
- Use QR codes for fast scanning, movement, checkout, return, and identification.
- Use AI-assisted photos to propose item details and storage locations.
- Track custody separately from location through checkout history.
- Support temporary transfers such as Garage to Trailer.
- Eventually support reusable activity/loadout checklists such as camping, skiing, or soccer.
- Remain usable when the companion phone app is offline.

Core design principle

Clients should not care where the server runs. The same web and mobile applications should communicate with the same versioned REST API whether the backend is running on a Raspberry Pi in a home or in the cloud.

Deployment modes

Mode	Backend	Storage
Self-hosted	FastAPI + PostgreSQL on Raspberry Pi	Local filesystem for photos
Cloud	Containerized FastAPI + PostgreSQL	S3-compatible object storage

2. Technology stack

Layer	Choice
Web	React + Vite + TypeScript
Mobile companion	React Native + Expo + TypeScript
Backend	Python 3.13 + FastAPI
ORM	SQLAlchemy 2
Migrations	Alembic
Server database	PostgreSQL
Mobile offline database	SQLite via Expo
Auth/device credential storage	Expo SecureStore
API contract	FastAPI OpenAPI -> generated TypeScript client
Python tooling	uv, Ruff, Pytest
JavaScript tooling	pnpm workspaces, Vitest
Local deployment	Docker Compose + Caddy
Cloud web	Netlify-compatible static React build

Repository layout

```
wherehouse/  
├─ apps/
```

```

|   ├── web/
|   └── mobile/
└── backend/
    ├── packages/
    │   └── api-client/
    ├── deploy/
    │   ├── docker/
    │   ├── raspberry-pi/
    │   └── cloud/
    ├── docs/
    ├── docker-compose.yml
    └── README.md

```

3. Current implementation status

The repository has been initialized and the first backend domain slice has been implemented on feature/core-domain. Pull Request #1 is open against main.

- FastAPI application scaffold and health endpoint.
- Async SQLAlchemy session configuration.
- PostgreSQL UUID-based persistence.
- Alembic configuration and initial core-domain migration.
- Pydantic request and response schemas.
- Core /api/v1 endpoints for creating and listing initial entities.
- Docker Compose startup that runs migrations before starting the API.
- Schema-level tests for important validation rules.

Implemented domain entities

Entity	Purpose
Household	Named household/workspace.
User	Registered human identity.
HouseholdUser	Relationship between user and household; owner or borrower.
Area	Major physical location such as Garage, Attic, Trailer, Shed.
Zone	Subdivision of an area such as North Wall or Workbench.
Container	Physical holder or support such as bin, shelf, drawer, cabinet, rack.
ContainerPlacement	Places one container in/on/under/attached_to another.
Item	Inventory record with quantity and descriptive fields.
ItemPlacement	Current physical location of an item.

4. Domain rules that should be preserved

- Canonical IDs are UUIDs. Human-readable codes and QR identifiers are separate from UUIDs.
- Multiple users may be owners of the same household.
- Borrowers are external-to-household users with intentionally limited access.
- A Trailer is normally modeled as an Area, not as a Container.
- A Container can hold or support another object. It does not need to be enclosed.
- Container placement relationship semantics matter: in, on, under, attached_to.
- Containers have an is_out_of_space flag that is manually authoritative in MVP.
- Removing an item must not automatically clear is_out_of_space.
- An Item record can represent multiple interchangeable objects stored together through quantity.
- If part of a quantity moves elsewhere, split records or otherwise preserve one active placement per item record.
- AI proposes inventory data; user-approved data is canonical.
- Checkout answers who has an item. Transfer answers where the item is.
- Server PostgreSQL is canonical. Mobile SQLite is a cache/replica plus offline operation queue.

Important future distinction

WhereHouse will likely distinguish an item's normal home from its current location. This enables temporary Garage-to-Trailer transfers while still remembering where the item should be returned.

5. Next recommended development slice

Before building substantial UI, implement authentication, server/device pairing, and the foundations needed for an offline companion app.

Authentication

- Keep authentication in FastAPI rather than depending on a cloud-only auth provider.
- Start with email/password or another simple local-capable strategy.
- Use short-lived access tokens plus refresh/device credentials.
- Keep authorization household-scoped.

Device model

Device

- id: UUID
- household_id: UUID
- user_id: UUID
- name: string
- type: phone | tablet | scanner | browser | other
- last_seen_at?: datetime
- is_active: bool
- created_at: datetime
- revoked_at?: datetime

Pairing flow

- User opens WhereHouse web app and chooses Settings -> Pair Device.
- Server creates a short-lived, single-use PairingSession.
- Web UI displays a QR code containing instance metadata and a one-time pairing token.
- Companion scans the QR code.
- Mobile exchanges the pairing token for device-specific credentials.
- Credentials are stored in SecureStore, not SQLite.
- The companion downloads the initial household dataset into SQLite.

Pairing QR payload concept

```
{
  "version": 1,
  "instanceId": "<uuid>",
  "baseUrl": "https://wherehouse.local",
  "pairingToken": "<short-lived-random-token>"
}
```

The server should store only a hash of the pairing token. Pairing tokens should expire quickly and become invalid immediately after use.

6. Offline-first companion architecture

Offline support is a core architectural requirement, not a later enhancement. The companion should continue working in garages, attics, trailers, storage units, and campsites with poor connectivity.

- Cache household inventory in mobile SQLite.
- Allow scanning, searching, adding, editing, moving, checkout/return, and transfer actions offline.
- Record changes as PendingOperations.
- Push operations when connectivity returns.
- Pull server changes using a sync cursor.
- Use UUIDs so records can be created offline without a server round trip.
- Start with simple conflict handling. Surface physical-location conflicts instead of silently overwriting them.

Conceptual sync API

POST /api/v1/sync/push

GET /api/v1/sync/pull?cursor=<cursor>

PendingOperation concept

PendingOperation

- id: UUID

- user_id: UUID

- household_id: UUID

- operation_type: string
- entity_id: UUID
- payload: JSON
- created_at: datetime
- synced_at?: datetime

7. Features planned after the pairing foundation

Feature	Intent
QR identifiers + labels	Opaque short IDs for items/containers, PDF label generation, reprints.
Photos + AI item capture	Photo-first item creation with observed/inferred/unknown distinction.
AI placement suggestions	Rank household-specific storage locations while respecting full containers.
Checkout / return	Who has what, who performed the transaction, dates, quantity, return location.
Transfers	Garage-to-Trailer style load/unload workflow, including entire movable containers.
Activity templates	Camping, ski weekend, soccer, road trip, etc. with inventory requirements and tasks.
Scanner support	Keyboard-style barcode/QR scanner workflows without redesigning the domain.

8. MVP boundary

The MVP should prove one complete household workflow rather than every future feature.

MVP includes

- Household and basic user relationships.
- Areas, zones, containers, nested placement.
- Items with quantity, category/tags/photos, and current placement.
- QR identifiers and printable labels.
- Web setup/search/bulk management.
- Mobile scanning, adding, moving, checkout/check-in.
- Basic offline cache and operation queue.
- Pairing the companion to either a Pi-hosted or cloud-hosted WhereHouse server.
- Basic AI item creation and basic AI placement suggestion.
- Simple transfer workflow.

Explicitly not required for the first MVP

- Sophisticated activity templates and learned camping suggestions.
- Weather-aware recommendations.
- External contacts who do not have accounts.
- NFC.

- Dedicated scanner-gun UI.
- Visual garage maps.
- Warranty/purchase/insurance tracking.
- Automatic container capacity estimation.
- Complex distributed replication between Pi and cloud instances.

9. How to start in VS Code with Codex

Recommended local setup:

```
git clone https://github.com/Jamesbillard12/wherehouse.git
cd wherehouse
git checkout feature/core-domain
code .
```

First inspect and run the current branch. Fix issues before adding another feature. Once PR #1 is validated and merged, create a focused branch for authentication and pairing.

```
git checkout main
git pull
git checkout -b feature/auth-pairing
```

Suggested Codex kickoff prompt

You are continuing development of the WhereHouse application.

Start by reading:

- README.md
- docs/architecture.md
- docs/domain-model.md
- docs/mvp.md
- this handoff document

Then inspect the current implementation on feature/core-domain and PR #1.

Tasks:

1. Run the backend tests and migrations locally.
2. Review the core-domain implementation for correctness.
3. Fix any issues before merging.
4. Implement authentication, Device, PairingSession, and companion-to-server pairing.
5. Preserve Raspberry Pi self-hosting and cloud deployment compatibility.
6. Preserve the offline-first mobile architecture.
7. Keep the backend a modular monolith.
8. Do not redesign domain rules without updating the project docs and explaining the reason.
9. Use focused feature branches and tests for each slice.

10. Engineering guardrails

- Prefer a modular monolith. Do not introduce microservices, Kafka, Kubernetes, or an event bus for the MVP.
- Do not make web/mobile clients depend directly on PostgreSQL or cloud vendor tables.
- Keep storage behind an abstraction so Pi uses filesystem and cloud can use S3-compatible storage.
- Keep AI behind a provider abstraction so inventory code does not directly call one provider.
- Generate TypeScript API types/client from FastAPI OpenAPI rather than duplicating contracts manually.
- Use migrations for every database schema change.
- Validate household ownership and cross-household references at API/service boundaries.
- Avoid hidden destructive behavior. Moves, transfers, checkout, and return should preserve useful history.
- Optimize for practical household use, especially scan-first mobile workflows.

11. Golden end-to-end flow

1. Create a household.
2. Create Garage area.
3. Create North Wall zone.
4. Create Shelf container.
5. Create Bin container and place it on Shelf.
6. Generate and print the Bin QR label.
7. Pair the companion phone to the WhereHouse server.
8. Scan the Bin QR code.
9. Photograph an item.
10. AI proposes the item fields; user confirms them.
11. Save the item into the scanned Bin.
12. Search for the item later and show its exact nested location.
13. Scan and transfer the item to Trailer.
14. Return it later to its normal home.
15. Check an item out to another user and preserve checkout history.

12. Current handoff checkpoint

The project is ready to move from architecture/scaffolding into normal iterative development in VS Code. The immediate checkpoint is to validate PR #1 locally, merge it, then build authentication and pairing as the next focused slice.